Experiment 8: K-Means Clustering

Francis Reuben R

Department of Computer Science and Engineering, RV University

Data Science Laboratory, Semester 3, Section G | USN: 1RUA25CSE0168 | 30 October 2026

Objective

Cluster points by moving centroids until each point sits with the nearest mean. The notebook works seven points by hand, clusters 22 people by age and income before and after scaling, rebuilds that clustering in NumPy, and then clusters 178 wines from 13 chemical measurements with a from-scratch K-Means and with scikit-learn.

Description

K-Means picks K centroids, assigns every point to the nearest centroid by Euclidean distance, and replaces each centroid with the mean of its points. Those two steps repeat until the assignment stops changing. Each step can only lower, or hold, the within-cluster sum of squares (WCSS), which scikit-learn stores as inertia_. The minimum is local, so the start matters. A tie goes to the first centroid, which is what NumPy argmin does.

The hand example has seven points, A through G, and K = 2. The start is A at (1, 1) and D at (5, 7). The code prints every distance, the assignment, the updated centroids, and the WCSS. It also computes the silhouette of point A and the mean silhouette of the seven points.

income.csv has 22 rows and three columns: Name, Age, and Income($). Age runs from 26 to 43. Income runs from 45,000 to 162,000. The first model uses K = 3 on the raw columns, with random_state 42 and scikit-learn's default single k-means++ start. Min-max scaling then puts both columns on [0, 1], and K = 3 is fit again with random_state 42 and n_init 10. SSE and silhouette are recorded for K = 2 to 9 on that scaled table. A separate NumPy K-Means, seed 42, is fit on the z-scored columns.

sklearn.datasets.load_wine supplies 178 rows and 13 chemical features. The cultivar counts are 59, 71, and 48 (class_0, class_1, class_2). The cultivar is not used to fit the model. It is used only for the adjusted Rand index and the normalised mutual information. StandardScaler is fit on all 13 features. PCA with two components is used for the scatter plot and is not the matrix that gets clustered. K runs from 2 to 10 in both implementations. The reported model is K = 3.

Flowchart

Figure 1 is the pipeline in Ex.No.8.ipynb. The charts from that notebook are Figures 2 to 8.

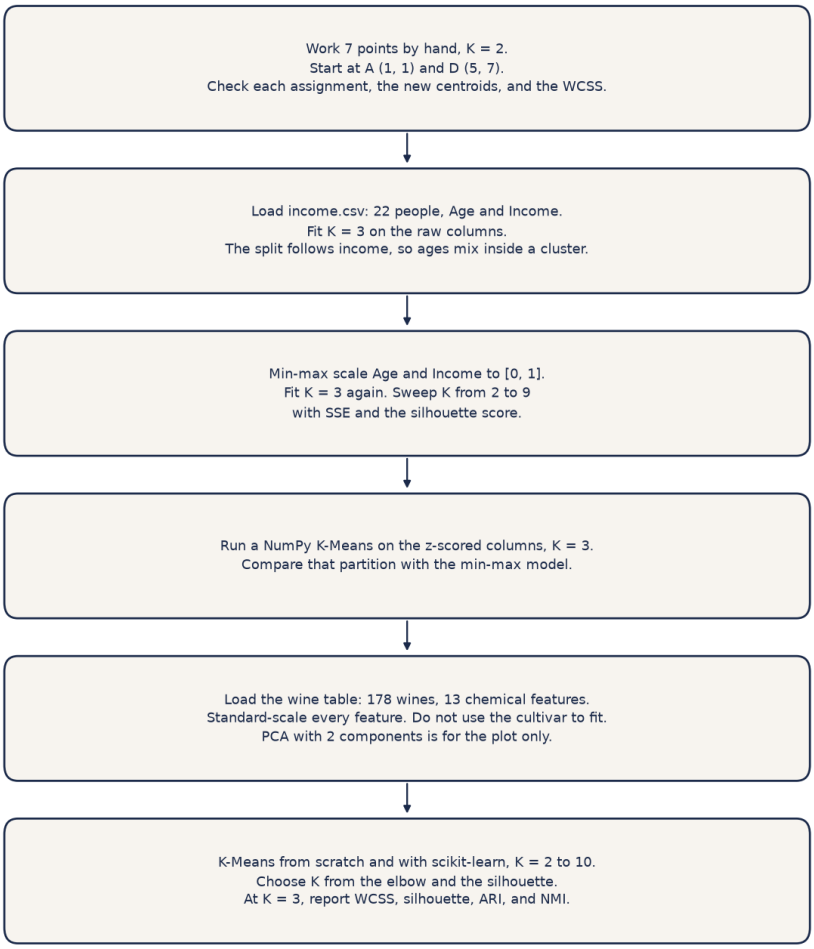


Figure 1. Pipeline for the hand example, the income models, and the wine comparison.

Algorithm

1. Start. Import pandas, NumPy, matplotlib, seaborn, and scikit-learn.
2. On the seven points, start at A (1, 1) and D (5, 7). Assign, update, and print the WCSS until the assignment stops changing. Print the silhouette of A and the mean silhouette.
3. Load income.csv. Fit KMeans with K = 3 and random_state 42 on raw Age and Income. Print the labels, the centroids, and the names in each cluster.
4. Min-max scale Age and Income. Fit K = 3 again with random_state 42 and n_init 10. Plot the clusters.
5. For K = 2 to 9 on the scaled columns, store inertia_ and silhouette_score. Plot both and print the table.
6. Z-score Age and Income. Run the NumPy K-Means at K = 3 with seed 42. Print the WCSS and the adjusted Rand index against the min-max labels.
7. Load the wine table. Print the shape, the head, and describe(). Standard-scale the 13 features. Fit PCA with two components for the plot only.
8. Write kmeans_scratch with random starts, n_init 10, and the lowest WCSS. Write a NumPy silhouette and compare it with sklearn.metrics.silhouette_score.

9. Fit sklearn KMeans at K = 3 with n_init 10 and random_state 42. Print inertia_, n_iter_, and the centroid shape.
10. For K = 2 to 10, plot WCSS and silhouette for both implementations. At K = 3, print WCSS, silhouette, ARI, and NMI, then the crosstabs against the cultivar.
11. End.

Worked example

Point B is the sample distance in the notebook. From (1.5, 2) to (1, 1) is 1.118. From (1.5, 2) to (5, 7) is 6.103. B joins the first centroid. Point C is the tie: both distances are 3.606, so C joins the first centroid on iteration 1 and leaves it on iteration 2.

Iteration	Centroids used for the assignment	WCSS
1	(1, 1) and (5, 7)	33.250
2	(1.833, 2.333) and (4.125, 5.375)	11.226
3	(1.25, 1.5) and (3.9, 5.1)	8.525

The assignment on iteration 3 is the same assignment that produced those centroids, so the run stops. C1 is {A, B}. C2 is {C, D, E, F, G}. The WCSS falls at every step. It does not rise.

For point A, the other member of C1 is B, so a(A) is the distance 1.118. The mean distance from A to the five points in C2 is 5.030. The silhouette is (5.030 - 1.118) / 5.030 = 0.778. The mean silhouette of all seven points is 0.645. Figure 2 shows the two final groups and the centroids.

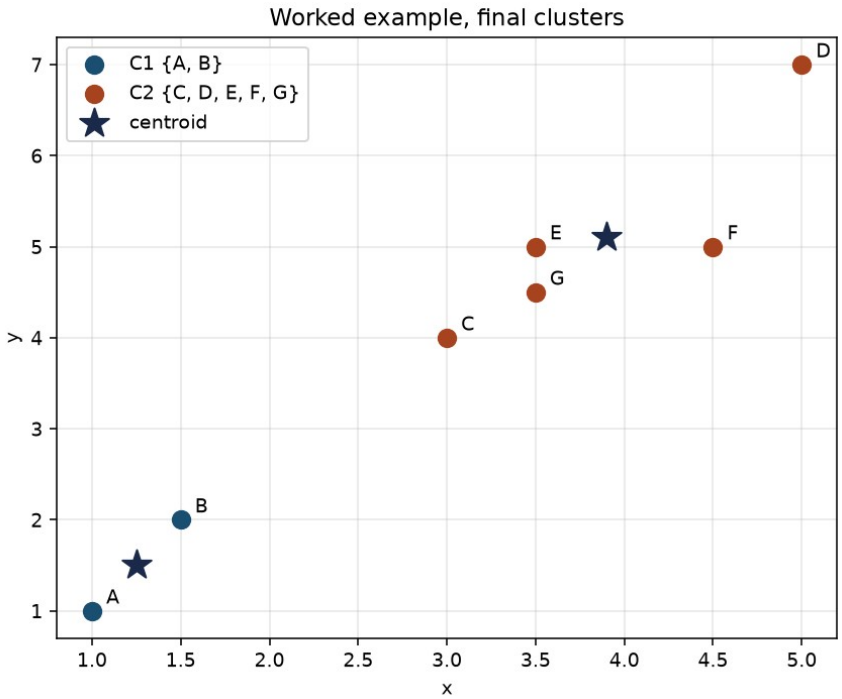


Figure 2. Final clusters of the seven-point example. C1 is A and B. C2 is C, D, E, F, and G.

Income, before scaling

On the raw columns the three centroids are (38.29 years, 150,000), (32.42 years, 57,292), and (36.33 years, 84,000). Inertia is 1,606,229,737.68. The number is large because the squared errors are in dollars. Figure 3 is

that fit. Green is the high-income group. Red is the lower-income group. Black is the three-person group. The stars are the centroids.

Cluster	n	Mean age	Mean income
0, high income	7	38.29	150,000
1, lower income	12	32.42	57,292
2, mid income	3	36.33	84,000

Cluster 1 puts Rob (27 years, 70,000) with Nick (43 years, 64,000). Cluster 2 is Michael (29, 90,000), Alia (39, 80,000), and Sid (41, 82,000). Age hardly moves the distance: a gap of 16 years is 16, and a gap of 6,000 dollars is 6,000. The clusters are income bands.

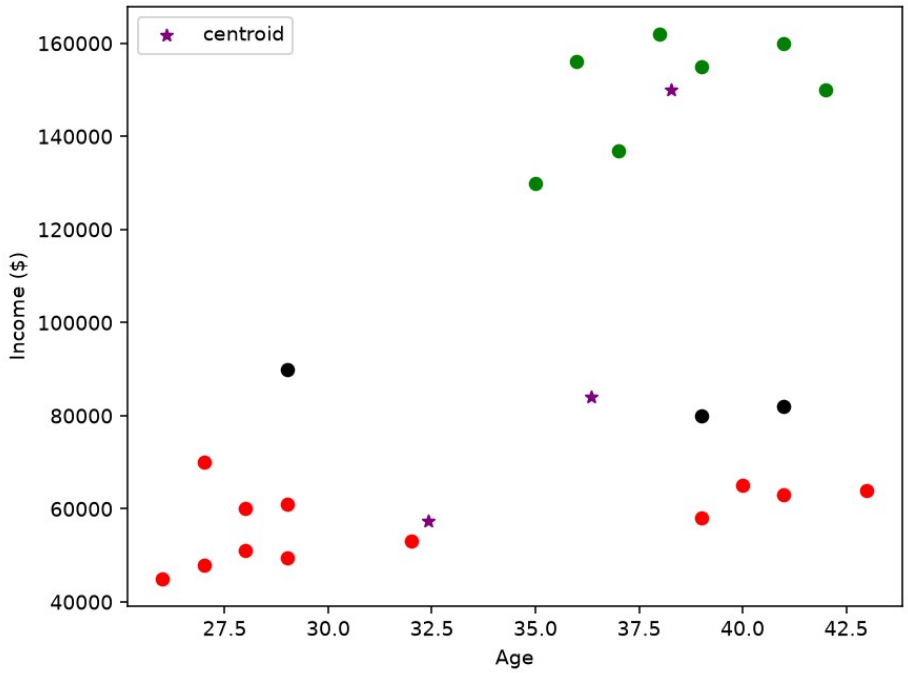


Figure 3. K = 3 on raw Age and Income. Colour follows the income band, not the age band.

After min-max scaling

Min-max scaling uses x' = (x - xmin) / (xmax - xmin). For Rob, age 27 becomes (27 - 26) / (43 - 26) = 0.058824, and income 70,000 becomes (70,000 - 45,000) / (162,000 - 45,000) = 0.213675. Both columns now lie on [0, 1], so a step in age counts as much as a step in income.

K = 3 on the scaled columns has inertia 0.4751. The three groups are the ones that were mixed in Figure 3. Michael moves in with the younger salaries. Alia and Sid move in with the older middle salaries. The high-income group is unchanged.

Cluster	n	Mean age	Mean income
0, high income	7	38.29	150,000
1, young	9	28.33	58,611
2, older, middle income	6	40.50	68,667

Means are in the original units. Figure 4 plots the scaled columns. Green is cluster 0, red is cluster 1, and black is cluster 2. The point sitting above the red centroid is Michael: his scaled income is 0.385 and his scaled age is 0.176, so he stays with the young group.

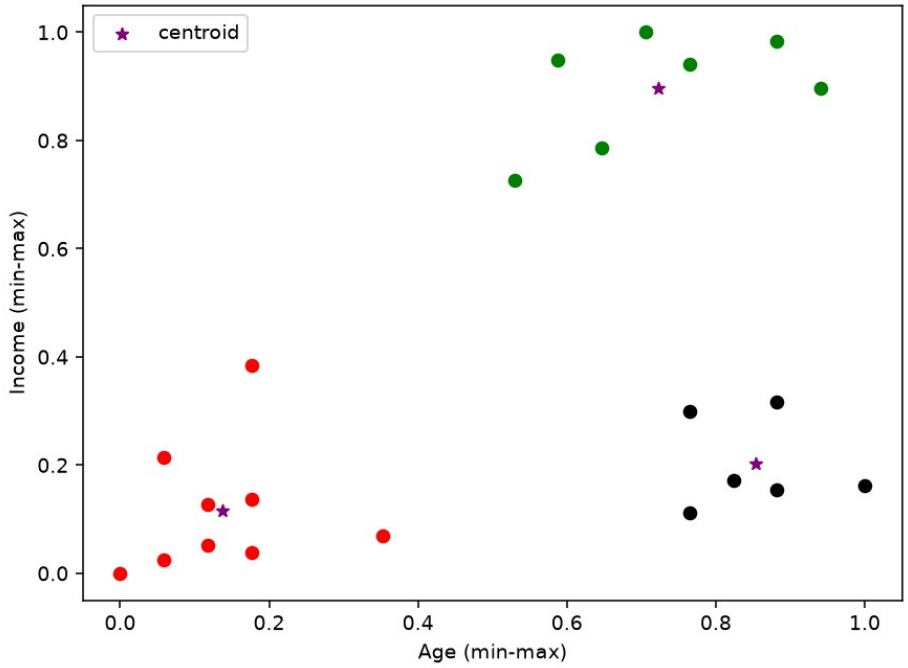


Figure 4. K = 3 after min-max scaling. The three groups separate on both axes.

Choosing K

Figure 5 is the sweep on the scaled columns. SSE drops from 2.091 at K = 2 to 0.475 at K = 3, and the slope is flatter after that. The silhouette is highest at K = 3.

K	SSE	Silhouette
2	2.0911	0.5601
3	0.4751	0.7130
4	0.3491	0.6257
5	0.2818	0.4680
9	0.1038	0.3298

K = 6, 7, and 8 sit on the same decline: silhouette 0.4288, 0.4522, and 0.3580. The highest score is 0.7130 at K = 3, which is the K used above.

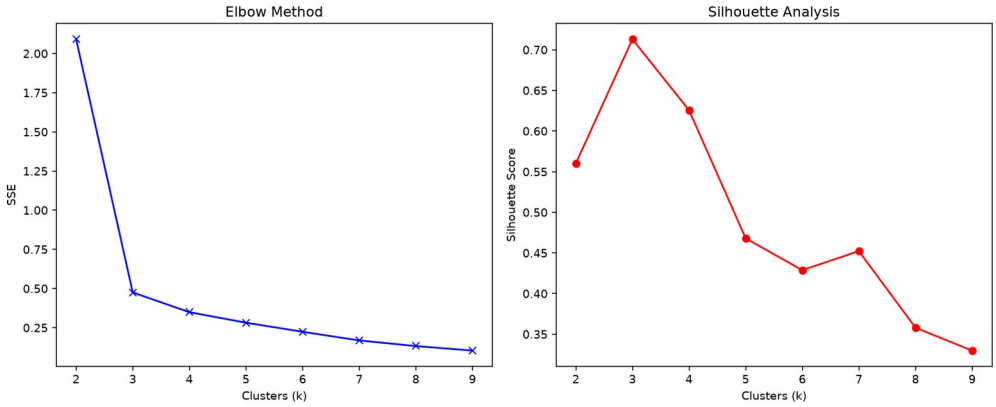


Figure 5. SSE and silhouette for K = 2 to 9 on min-max scaled Age and Income.

K-Means from scratch

The NumPy function draws K distinct rows as the first centroids, assigns by Euclidean distance, and replaces each centroid with the mean of its rows. Seed 42, K = 3, on the z-scored columns gives WCSS 3.8893. The

adjusted Rand index against the min-max labels is 1.0, so the two runs found the same partition. Cluster 0 is the 9 younger people, cluster 1 is the 7 high incomes, and cluster 2 is the 6 older middle incomes. The centroids in z-score units are (-1.125, -0.749), (0.601, 1.401), and (0.986, -0.512). Figure 6 is that fit.

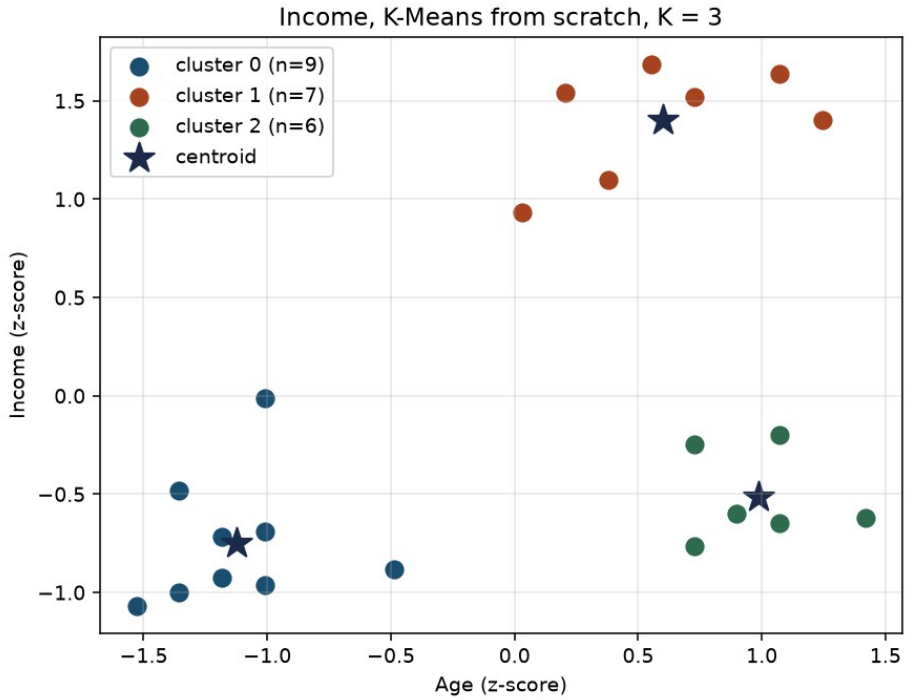


Figure 6. NumPy K-Means on z-scored Age and Income, K = 3. Same three groups as Figure 4.

Wine

The 13 features do not share a scale. Proline runs from 278 to 1,680. Hue runs from 0.48 to 1.71. Magnesium runs from 70 to 162. Alcohol runs from 11.03 to 14.83. Euclidean distance on the raw matrix would be a proline distance. StandardScaler gives each column mean 0 and standard deviation 1, and the clustering uses that 178 by 13 matrix. The two principal components explain 0.362 and 0.192 of the scaled variance, 0.554 together. They are used only in Figure 8.

K = 3

scikit-learn at K = 3, n_init 10, random_state 42, reaches inertia 1,277.928 in 7 iterations. The centroid matrix is 3 by 13. The from-scratch run, also n_init 10 and seed 42, reaches the same WCSS, 1,277.928. The silhouette is 0.2849, which the comparison table rounds to 0.285. A NumPy silhouette written for this lab matches sklearn.metrics.silhouette_score. The largest absolute gap on the 178 point scores is about 10^-15.

Metric	Better	Scratch	scikit-learn
WCSS / inertia	Lower	1,277.928	1,277.928
Silhouette	Higher	0.285	0.285
Adjusted Rand index	Higher	0.897	0.897
Normalised mutual information	Higher	0.876	0.876

The two label vectors are one partition written in two orders. In the scratch labeling, class_0 (59 wines) is entirely in cluster 2, class_2 (48 wines) is entirely in cluster 0, and class_1 (71 wines) has 65 wines in cluster 1. The other 6 wines of class_1 are split 3 and 3 into the neighbouring clusters. scikit-learn's cluster 0 is the scratch cluster 1, its cluster 1 is the scratch cluster 0, and its cluster 2 is the scratch cluster 2.

Cultivar	Scratch 0	Scratch 1	Scratch 2	Rows
class_0	0	0	59	59
class_1	3	65	3	71
class_2	48	0	0	48

Choosing K

Figure 7 plots both implementations for K = 2 to 10. The silhouette peaks at K = 3 for both, at 0.2849. The WCSS drops from 1,658.759 at K = 2 to 1,277.928 at K = 3, and the next drop is much smaller. That is the elbow, and it matches the three cultivars. Past K = 3 the two WCSS curves separate by a few units. At K = 8 the scratch WCSS is lower (926.385 against 935.201). At the other K from 4 to 10 the sklearn WCSS is lower. The silhouette peak does not move.

K	Scratch WCSS	Scratch sil.	sklearn WCSS	sklearn sil.
2	1,658.759	0.2593	1,658.759	0.2593
3	1,277.928	0.2849	1,277.928	0.2849
4	1,180.247	0.2504	1,175.428	0.2602
5	1,111.425	0.2365	1,109.513	0.2016
10	868.666	0.1378	845.895	0.1436

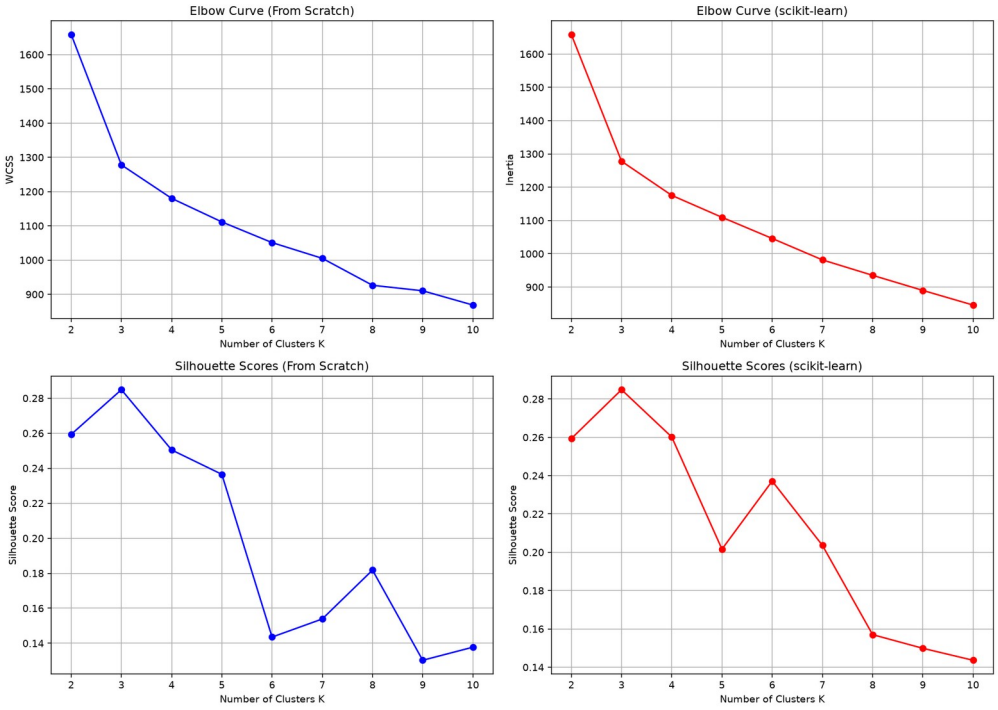


Figure 7. WCSS and silhouette for K = 2 to 10. Left is the NumPy K-Means. Right is scikit-learn.

Figure 8 draws the same two principal components three times. The left panel is the cultivar. The middle panel is the scratch labeling. The right panel is scikit-learn. The three clouds are the same. The colours differ because each panel numbers its groups on its own. The six class_1 wines that the crosstab places outside the main cluster are the points that sit against a neighbouring cloud.

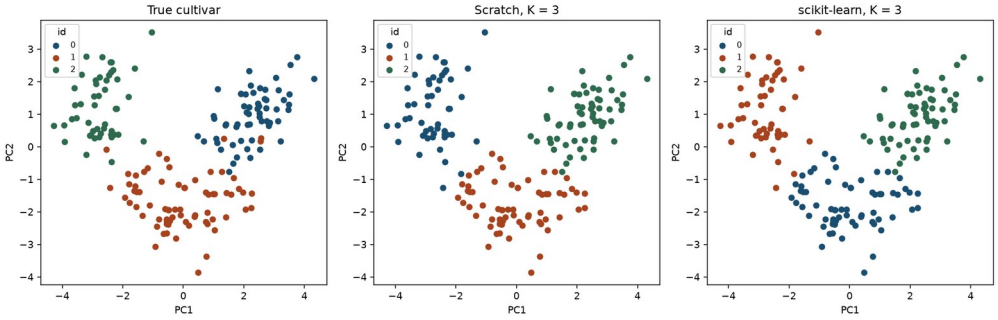


Figure 8. Wine, first two principal components. Clustering used all 13 scaled features. PCA is the display.

Result

The seven-point example converges in three iterations. The centroids finish at (1.25, 1.5) and (3.9, 5.1), the clusters are {A, B} and {C, D, E, F, G}, and the WCSS path is 33.250, 11.226, 8.525. Point A has silhouette 0.778. On the income table, K = 3 fit to raw Age and Income groups people by salary: a 12-person cluster mixes ages 26 to 43. After min-max scaling the same K separates a young group of 9 (mean age 28.33, mean income 58,611), a high-income group of 7 (mean income 150,000), and an older middle group of 6 (mean age 40.50, mean income 68,667). The silhouette on the scaled columns is highest at K = 3 (0.7130). The NumPy K-Means on the z-scored columns recovers that same partition, adjusted Rand index 1.0, with WCSS 3.8893. On the wine table both implementations at K = 3 reach WCSS 1,277.928, silhouette 0.285, ARI 0.897, and NMI 0.876. The silhouette peaks at K = 3, which is the number of cultivars. Six of the 71 class_1 wines fall into the other two clusters. The from-scratch silhouette matches scikit-learn to about 10^-15.

GitHub

Course repository: https://github.com/francisreubenr-rvu/Data-Science-Labs

Lab 08 is the folder Semester-3/Labs-Semester-3/Lab 08 in that repository.

The executed notebook is Ex.No.8.ipynb in that folder. The upload copy is 1RUA25CSE0168_Francis_Reuben_R_Lab8.ipynb. Outputs are saved in the file.